

Proyecto De Fin De Grado

Juan Bustos Espinosa

24 de marzo de 2025

1. Introducción

En la actualidad, las redes sociales han transformado la forma en que las personas se comunican, comparten intereses y forman comunidades en torno a temáticas específicas. Sin embargo, a pesar de la popularidad de los videojuegos y del gran número de jugadores activos en todo el mundo, no existe una red social dedicada exclusivamente a este ámbito, los videojuegos que combine tanto el seguimiento de lanzamientos, actualizaciones de los juegos como la interacción centrada únicamente en el contenido de videojuegos. Este proyecto tiene como objetivo el desarrollo de una red social especializada en videojuegos, que sirva como punto de encuentro para jugadores y empresas del sector.

La plataforma está diseñada para fomentar una comunidad donde toda la actividad gire en torno al mundo de los videojuegos. A diferencia de otras redes sociales más generalistas, esta red limita las publicaciones a contenido exclusivamente relacionado con videojuegos, lo que garantiza un entorno temático coherente y libre de ruido informativo. Los usuarios pueden publicar mensajes, comentar a otros usuarios para dar su opinión o mantener una conversación sobre un videojuego, mantenerse informados sobre los títulos más recientes del mercado o futuras actualizaciones para tu videojuego favorito.

La red social contempla dos tipos de usuarios: usuarios particulares y empresas desarrolladoras o distribuidoras de videojuegos. Mientras que los usuarios pueden interactuar entre sí mediante publicaciones y comentarios, las empresas tienen el rol de publicar nuevos videojuegos, generar contenido promocional y también participar en las conversaciones. Esta diferenciación de roles permite una interacción directa entre creadores y consumidores, fomentando un espacio participativo, informativo y enfocado al entretenimiento digital.

El objetivo principal de esta red social es facilitar una comunicación más abierta entre usuarios y empresas. De este modo, las empresas pueden recibir retroalimentación directa por parte de los jugadores, lo que les permite identificar mejoras potenciales, comprender las necesidades de la comunidad y reaccionar ante críticas constructivas. Esto no solo beneficia a las empresas, que pueden perfeccionar sus productos, sino también a los usuarios, que se sienten escuchados y partícipes del proceso de desarrollo. Este proyecto no solo aborda el desarrollo técnico de la plataforma, sino también el análisis de necesidades del

sector, la arquitectura de la aplicación, la gestión de roles y contenidos, y los mecanismos para garantizar una experiencia de usuario segura, fluida y centrada en los intereses reales de la comunidad gamer.

2. Justificación y Objetivos del Proyecto

Justificación

La industria de los videojuegos ha experimentado un crecimiento exponencial en las últimas décadas, convirtiéndose en uno de los sectores de entretenimiento más relevantes a nivel global. Las redes sociales se han consolidado como algo esenciales para la interacción digital, el intercambio de opiniones y la creación de comunidades. No obstante, a pesar del enorme número de jugadores y empresas dedicadas al desarrollo de videojuegos, no existe actualmente una red social centrada exclusivamente en esta temática que ofrezca una experiencia segmentada, ordenada y enfocada solo en el contenido gamer.

Las plataformas actuales, como Twitter(ahora conocido como X) o Reddit, aunque permiten la discusión sobre videojuegos, presentan contenido muy diverso, lo que puede dificultar la interacción focalizada en este ámbito. Además, no suelen ofrecer herramientas específicas para conectar a usuarios con empresas desarrolladoras, ni estructuras diseñadas para el seguimiento detallado de videojuegos, sus lanzamientos, actualizaciones o feedback en tiempo real.

Este proyecto surge precisamente para cubrir ese vacío. La propuesta de una red social exclusiva para videojuegos responde a la necesidad de contar con un entorno digital que reúna a jugadores y empresas en un mismo espacio, con reglas claras de publicación, centradas únicamente en el contenido del sector. De esta manera, se potencia una comunidad activa, informada y participativa, donde las empresas pueden recibir opiniones útiles directamente de sus consumidores y los usuarios pueden mantenerse actualizados y conectados con sus intereses reales.

Objetivos

Objetivo general

Desarrollar una red social especializada en videojuegos que permita la

interacción entre usuarios y empresas, centrada exclusivamente en contenido gamer, con funcionalidades que faciliten la publicación, seguimiento, discusión y actualización de videojuegos.

Objetivos específicos

- Diseñar una estructura de usuarios diferenciada (usuarios particulares y empresas) con permisos y funcionalidades adaptadas a cada rol.
- Implementar un sistema de publicaciones limitado únicamente a contenido relacionado con videojuegos, garantizando coherencia temática.
- Crear un módulo de gestión de videojuegos donde las empresas puedan dar de alta títulos, actualizaciones y noticias relacionadas.
- Desarrollar herramientas de interacción social, como comentarios y seguimiento de juegos o usuarios.
- Garantizar una experiencia de usuario fluida, intuitiva y segura, priorizando el diseño responsive y accesible.
- Fomentar un entorno participativo donde los jugadores puedan influir con su feedback en el desarrollo y mejora de videojuegos.

¿Por qué hacemos este proyecto?

Hacemos este proyecto porque actualmente no existe una red social dedicada exclusivamente al mundo de los videojuegos que combine de forma efectiva la interacción entre jugadores y empresas con el seguimiento de lanzamientos, actualizaciones y contenido relacionado únicamente con videojuegos. A pesar de la enorme popularidad del sector, los jugadores suelen estar dispersos en redes sociales generalistas donde el contenido no siempre está enfocado, lo que limita la experiencia y la calidad de la interacción.

Este proyecto busca llenar ese vacío, ofreciendo una plataforma donde se

priorice la temática gamer, se fomente una comunidad especializada y se facilite la comunicación directa entre usuarios y empresas desarrolladoras o distribuidoras. Con ello, no solo se mejora la experiencia de los jugadores, que encuentran un entorno centrado en sus intereses, sino que también se proporciona a las empresas una herramienta útil para recibir feedback real y mejorar sus productos.

3. Metodología

A. Planificación de las tareas del proyecto

El trabajo se ha estructurado en varias fases, cada una con tareas específicas y objetivos claros:

1. Análisis de requisitos

En esta fase inicial se ha realizado un estudio detallado para definir qué funcionalidades debía tener la aplicación, tanto desde el punto de vista del usuario como de la empresa.

Se han recogido los requisitos funcionales, como la posibilidad de registrarse, publicar mensajes, comentar, ver juegos, y distinguir entre tipos de usuarios (usuarios y empresas).

También se han considerado los requisitos no funcionales, como la escalabilidad, la seguridad de los datos, la usabilidad, el rendimiento y la disponibilidad.

2. Diseño de la arquitectura de la aplicación

Se ha definido una arquitectura basada en tres capas principales:

- **Frontend** (cliente) desarrollado con React.
- **Backend** desarrollado con Spring Boot.
- **Base de datos** con MySQL.

En esta fase también se ha planificado la estructura de carpetas, las tecnologías específicas a utilizar, la seguridad (como autenticación y autorización), y la manera en que los distintos componentes se relacionan entre sí.

3. Desarrollo de la base de datos

Se ha creado el modelo relacional de datos en MySQL, incluyendo la definición de tablas, relaciones, claves primarias y foráneas. Algunas de las entidades diseñadas han sido: usuario, mensaje, juego...

Se ha cuidado la normalización de los datos para evitar redundancias y asegurar la integridad referencial.

4. Desarrollo del backend con Spring Boot

En esta etapa se ha desarrollado toda la lógica del servidor usando el framework Spring Boot.

Las principales tareas incluyeron:

- Creación de controladores.
- Servicios que implementan la lógica de negocio.
- Repositorios para el acceso a la base de datos usando Spring Data JPA.
- Implementación de seguridad (autenticación y autorización).
- Manejo de errores y validación de entradas.

El backend actúa como intermediario entre la base de datos y el frontend, gestionando las peticiones del cliente y enviando respuestas estructuradas.

5. Desarrollo del frontend con React

En esta fase se ha desarrollado la interfaz de usuario con React, organizando la aplicación en componentes reutilizables.

Las funcionalidades incluidas abarcan:

- Registro e inicio de sesión.

- Vista de publicaciones, videojuegos y tendencias.
- Creación y gestión de publicaciones, comentarios y videojuegos.
- Distinción visual entre usuarios y empresas.
- Consumo de la API REST con herramientas como Axios.

6. Integración del sistema (frontend + backend + base de datos)

Una vez desarrolladas las partes por separado, se ha realizado la integración del sistema.

Esto ha implicado:

- Asegurar que el frontend se comuniquen correctamente con el backend.
- Configurar CORS para permitir el acceso desde el cliente.
- Verificar que las respuestas del backend sean correctamente interpretadas por el frontend.
- Confirmar que la base de datos responda de manera eficiente a las consultas.

7. Pruebas funcionales

Se han llevado a cabo pruebas para comprobar que cada funcionalidad del sistema funcione según lo definido en los requisitos.

Estas pruebas han incluido:

- Comprobación de formularios de registro/login.
- Publicación y visualización de mensajes.
- Publicación y visualización de juegos
- Validación de entradas (por ejemplo, evitar Nickname repetidos).
- Comprobación de restricciones de acceso según el tipo de usuario.

8. Documentación

A lo largo del desarrollo del proyecto se ha elaborado una memoria detallada que recoge de forma estructurada todos los aspectos relevantes del mismo. Esta documentación no solo sirve como soporte académico, sino también como guía técnica que facilita la comprensión, evaluación y posible evolución futura del sistema.

La memoria incluye los siguientes apartados principales:

- **Introducción al proyecto**, donde se presenta la motivación, el contexto y los objetivos de la red social de videojuegos desarrollada.
- **Justificación del proyecto**, explicando por qué se considera necesaria esta aplicación, qué necesidad resuelve y cuál es su aportación dentro del sector.
- **Tecnologías utilizadas**, con una explicación razonada de la elección de herramientas como React, Spring Boot, MySQL, entre otras.
- **Descripción de funcionalidades**, detallando las características implementadas tanto para usuarios particulares como para empresas (publicaciones, comentarios, gestión de juegos, etc.).
- **Planificación y gestión de tareas**, presentando la metodología de trabajo, el desglose de actividades, el diagrama de Gantt y el seguimiento del progreso.
- **Diseño técnico**, que incluye diagramas de arquitectura, modelo entidad-relación de la base de datos, y estructura de la API.
- **Metodología de pruebas**, con los métodos utilizados para comprobar el correcto funcionamiento del sistema.
- **Como se continuara**, se explica posibles cambios o cosas que se añadirían en caso de seguir con el proyecto
- **Conclusiones**, donde se reflexiona sobre el trabajo realizado, los aprendizajes adquiridos y posibles mejoras futuras.

9. Presentación

Como parte final del proyecto, se ha preparado una presentación con el objetivo de exponer de forma clara, concisa y estructurada el trabajo realizado.

B. Tiempo estimado para cada tarea

Tarea	Tiempo
Análisis de requisitos	1
Diseño de arquitectura	2
Base de datos(MySQL)	1
Backend(Spring Boot)	2
Frontend(React)	3
Integración	1
Pruebas	2
Documentación	2
Presentación	1
Total Estimado	14 semanas

C. Responsable de cada tarea

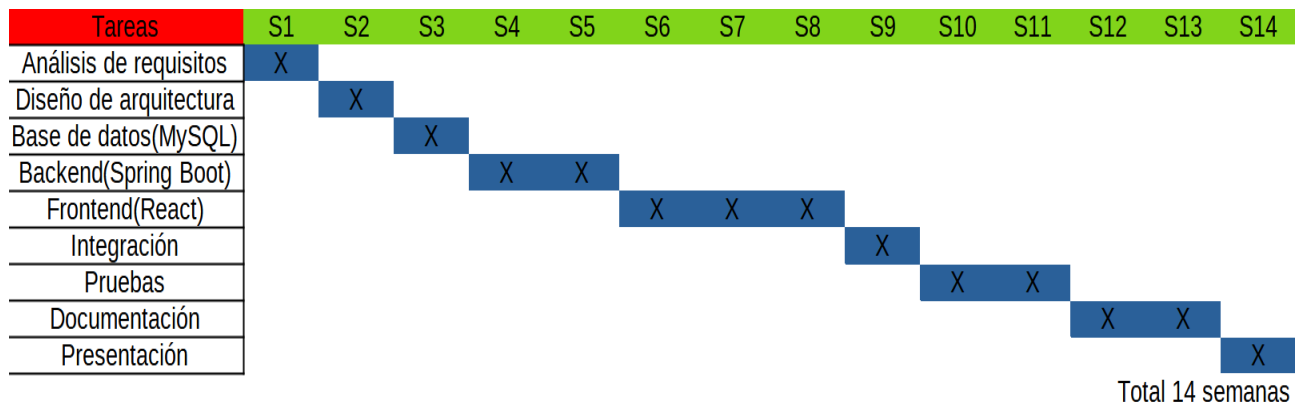
Todas las tareas han sido ejecutadas por el autor del proyecto, quien ha asumido los roles de analista, diseñador, desarrollador full stack, tester y redactor técnico.

D. Dependencias entre tareas

- El backend depende del diseño de arquitectura y del modelo de base de datos.
- El frontend puede desarrollarse en paralelo una vez definido el diseño de interfaces, pero necesita de la API para funcionalidades completas.
- La integración requiere la finalización del backend, base de datos y parte del frontend.
- Las pruebas dependen de la integración del sistema.

- La documentación puede iniciarse en paralelo, pero se completa al finalizar el desarrollo.

E. Diagrama de Gantt



F. Herramientas de trabajo de comunicación

-**Trello / Notion:** organización de tareas y progreso.

Aunque se trata de un proyecto individual, se ha utilizado Trello (o Notion) para organizar tareas, planificar las distintas fases del proyecto y llevar un control visual del progreso. Estas herramientas permiten una gestión ágil del trabajo, similar a metodologías como Scrum o Kanban, incluso en proyectos personales.

G. Revisión y control del progreso

Al ser un proyecto individual, se han llevado a cabo **revisiones semanales de seguimiento**, que consistían en sesiones de evaluación personal del avance respecto al plan inicial. Estas reuniones han permitido detectar bloqueos, reorganizar prioridades y ajustar los tiempos cuando ha sido necesario.

- **Periodicidad de revisión:** semanal (1 vez por semana).
- **Duración aproximada:** entre 30 minutos y 1 hora.
- **Objetivo:** revisar tareas completadas, pendientes, dificultades técnicas y próximos pasos.

4. Desarrollo del Proyecto

A. Herramientas principales y Funcionalidad principal

Para el desarrollo de la red social de videojuegos se han utilizado tecnologías modernas y ampliamente aceptadas en el desarrollo de aplicaciones web. A continuación, se detallan:

Herramienta / Tecnología	Función principal
React	Desarrollo del frontend, interfaz de usuario interactiva y dinámica.
Spring Boot	Backend y creación de API REST. Gestiona la lógica de negocio, seguridad y acceso a la base de datos.
MySQL	Sistema de gestión de base de datos relacional para almacenar usuarios, juegos, comentarios ...
Postman	Pruebas de endpoints del backend.
GitHub	Control de versiones y almacenamiento del repositorio remoto.
IntelliJ IDEA	IDE utilizado para desarrollar el backend con Spring Boot.
Visual Studio Code	Editor de código utilizado para el desarrollo del frontend.

B. ¿Por qué se ha decidido cada tecnología?

- **React**

Se ha elegido React para el desarrollo del frontend por ser una de las bibliotecas más populares y eficientes para construir interfaces de usuario modernas. Su enfoque basado en componentes reutilizables permite mantener un código limpio y escalable. Además, su ecosistema y comunidad activa facilitan el aprendizaje, el soporte y la integración con otras herramientas.

- **Spring Boot**

Spring Boot es un framework robusto y ampliamente utilizado para construir aplicaciones backend en Java. Su estructura modular, junto con el soporte nativo para APIs REST, lo hace ideal para separar la lógica de

negocio del frontend. También incluye herramientas de seguridad (Spring Security), gestión de dependencias (Maven/Gradle) y configuración sencilla.

- **MySQL**

Se optó por MySQL como sistema gestor de bases de datos por su rendimiento, fiabilidad y facilidad de integración con Spring Boot mediante JPA/Hibernate. Además, es gratuito, ampliamente documentado y adecuado para proyectos de tamaño medio como este.

- **Postman**

Postman permite realizar pruebas rápidas y efectivas sobre los endpoints del backend sin necesidad de implementar previamente el frontend. Es útil para verificar que las rutas de la API funcionan correctamente, validar datos y detectar errores.

- **GitHub**

GitHub ha sido elegido para el control de versiones, permitiendo llevar un seguimiento detallado del historial de cambios. Y en caso de pérdida de datos nos ayuda a tener un backup del código

- **IntelliJ IDEA**

Este entorno de desarrollo integrado está especialmente optimizado para trabajar con proyectos Java y Spring Boot. Ofrece funciones como autocompletado inteligente, integración con bases de datos, gestión de dependencias y depuración eficiente.

- **Visual Studio Code**

Es un editor ligero, rápido y altamente personalizable que se adapta muy bien al desarrollo con React. Su integración con Github, su sistema de extensiones y su buena experiencia de usuario.

C. Diseño o esquemas de las tecnologías del proyecto

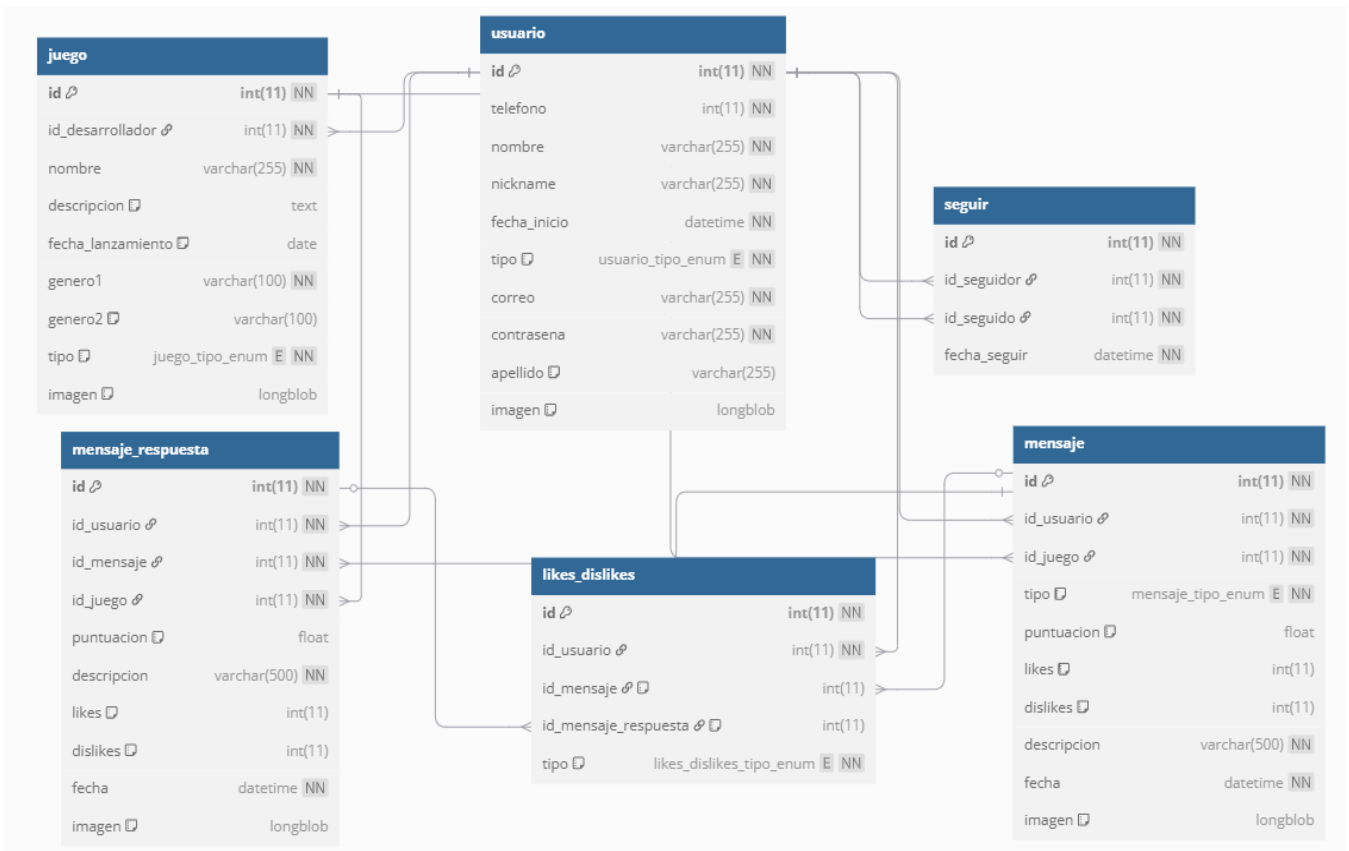


La arquitectura de la red social se basa en un modelo de tres capas: Frontend, Backend y Base de datos. Cada capa cumple un rol específico y se comunica con la siguiente mediante peticiones y respuestas. A continuación se muestra un esquema visual de cómo se relacionan:

Descripción del flujo:

1. El frontend (desarrollado en React) envía una petición HTTP al backend solicitando datos (por ejemplo, lista de videojuegos o creación de una publicación).
2. El backend (desarrollado con Spring Boot) procesa la solicitud, valida los datos y envía una consulta a la base de datos.
3. La base de datos (MySQL) responde al backend con los datos solicitados.
4. El backend devuelve al frontend el resultado de la petición, ya sea datos, confirmaciones o mensajes de error.
5. El usuario visualiza los datos o mensajes a través de la interfaz gráfica.

D. Base de datos



Usuario

Contiene los datos personales de cada persona registrada en el sistema, sean usuarios comunes o empresas desarrolladoras.

Campos:

- tipo — Tipo de usuario ("usuario" o "empresa")

Juego

Representa un videojuego en la plataforma, desarrollado por una empresa (usuario tipo empresa).

Relación:

- 1:N juego.id_desarrollador → usuario.id (Muchos juegos pueden ser creados por una empresa (usuario)).

Mensaje

Los mensajes que se pueden publicar sobre un juego

Campos:

- tipo — "futuro", "informe" o "critica"

Relaciones:

- 1:N → mensaje.id_usuario → usuario.id
- 1:N → mensaje.id_juego → juego.id
- 1:N → mensaje_respuesta.id_mensaje → mensaje.id

Mensaje_respuesta

Respuestas a un mensaje principal.

Relaciones:

- 1:N → mensaje_respuesta.id_usuario → usuario.id
- 1:N → mensaje_respuesta.id_mensaje → mensaje.id
- 1:N → mensaje_respuesta.id_juego → juego.id

Likes_dislikes

Registra las reacciones de los usuarios a mensajes o respuestas.

Campos:

- id_mensaje — Mensaje al que reacciona (opcional, FK)
- id_mensaje_respuesta — Respuesta a la que reacciona (opcional, FK)
- tipo — "like" o "dislike"

Relaciones:

- 1:N → likes_dislikes.id_usuario → usuario.id

- 1:N → likes_dislikes.id_mensaje → mensaje.id
- 1:N → likes_dislikes.id_mensaje_respuesta → mensaje_respuesta.id

Seguir

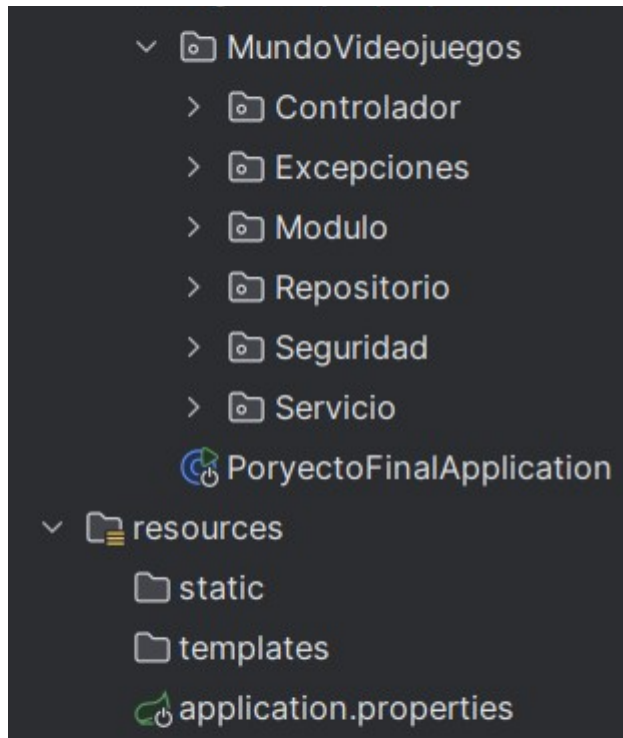
Representa que un usuario sigue a otro (como en redes sociales).

Relaciones:

- 1:N → seguir.id_usuario1 → usuario.id
- 1:N → seguir.id_usuario2 → usuario.id

Spring Boot

En el proyecto el Backend esta estructurado de la siguiente forma:



- **Controlador:** contiene los endpoints RESTful que reciben y responden a las peticiones del cliente (frontend).
- **Excepciones:** permite una gestión controlada de errores, mostrando mensajes personalizados a el error.
- **Modulo:** alberga las clases que mapean las tablas de la base de datos (por ejemplo: Usuario, Juego, Mensaje).
- **Repositorio:** usa interfaces extendidas de `JpaRepository` para operaciones CRUD sin necesidad de escribir SQL.
- **Seguridad:** contiene el archivo `ConfigSecurity` es el encargado de la gestion de la seguridad del servidor.
- **Servicio:** lógica de negocio, validaciones y coordinación entre controlador y repositorio.

- **application.properties:** configuraciones clave como el puerto del servidor y las credenciales de base de datos.

Librerías principales utilizadas

Dependencia	Descripción y motivo de uso
<code>Spring-boot-starter-web</code>	Incluye todo lo necesario para desarrollar aplicaciones web con <code>Spring Boot</code> . Es la base del backend de la API.
<code>spring-boot-starter-test</code>	Proporciona herramientas y dependencias para realizar pruebas unitarias y de integración en el proyecto. Se utiliza en el entorno de test.
<code>spring-boot-starter-security</code>	Añade funcionalidades de seguridad como autenticación, autorización, cifrado de contraseñas, etc. Es fundamental para proteger la aplicación y restringir el acceso.
<code>spring-boot-starter-actuator</code>	Proporciona endpoints para monitorizar y gestionar el estado de la aplicación. Útil durante el desarrollo y mantenimiento.
<code>jakarta.servlet-api</code>	Necesaria para manejar servlets, que permiten el procesamiento de peticiones web en el servidor. Se usa en modo provided ya que el servidor la proporciona en tiempo de ejecución.
<code>jackson-datatype-jsr310</code>	Permite que Jackson (librería de serialización/deserialización JSON) trabaje correctamente con tipos de fecha y hora de Java 8 (como <code>LocalDate</code> , <code>LocalDateTime</code> , etc.).
<code>hibernate-core</code>	Es el núcleo del ORM Hibernate, utilizado junto con JPA para mapear clases Java a tablas de base de datos. Permite persistencia de datos de forma sencilla.
<code>jaxb-runtime</code>	Necesaria para trabajar con XML si el proyecto tiene que generar, leer o transformar datos en formato XML.
<code>mariadb-java-client</code>	Driver JDBC necesario para conectar la aplicación con una base de datos MariaDB o MySQL. Gestiona la conexión entre <code>Spring Boot</code> y el sistema de base de datos.
<code>spring-boot-starter-data-jpa</code>	Simplifica el acceso a la base de datos mediante JPA y proporciona una forma rápida de trabajar con repositorios.
<code>spring-boot-starter-validation</code>	Añade soporte para validaciones automáticas en las clases (por ejemplo: <code>@NotNull</code> , <code>@Email</code> , <code>@Size</code>). Muy útil para verificar que los datos recibidos sean correctos.
<code>annotations (JetBrains)</code>	Añade anotaciones de ayuda para mejorar la legibilidad del código y facilitar la integración con IDEs como IntelliJ IDEA. No es esencial para la ejecución, pero mejora el desarrollo.

Modelo de datos: Clases entidad y mapeo con JPA

Cuando el frontend realiza una operación como crear un nuevo usuario, no siempre es necesario que envíe todos los campos. Por ejemplo, atributos como la fecha pueden ser gestionados directamente desde el backend, evitando así complicaciones innecesarias en el cliente.

Para ello, Spring Boot permite el uso de la anotación `@PrePersist`, que ejecuta un método justo antes de que una entidad se almacene por primera vez en la base de datos. Esto es útil para asignar automáticamente valores como fechas, estados iniciales, etc.

```
// Este método se ejecutará antes de persistir la entidad
@PrePersist
public void prePersist() {
    if (this.fechaInicio == null) {
        this.fechaInicio = LocalDateTime.now();
    }
}
```

Gestión de datos con Spring Data JPA

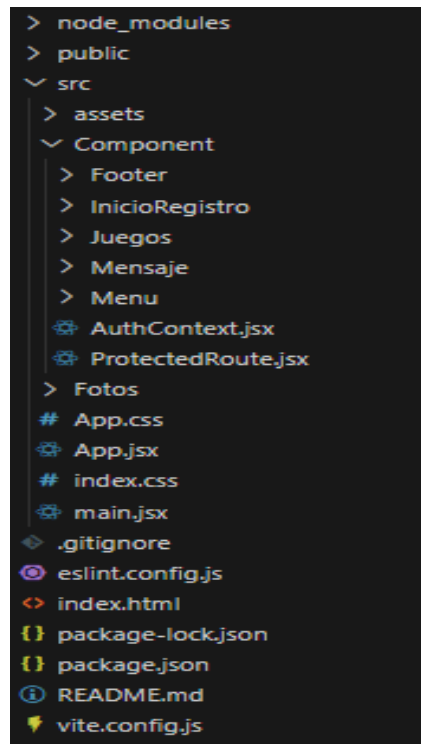
Además de las operaciones CRUD básicas que proporciona Spring Data JPA, en este proyecto ha sido necesario implementar consultas más específicas mediante la anotación `@Query`, que permite escribir consultas personalizadas en JPQL (Java Persistence Query Language).

Un caso representativo es la necesidad de obtener todos los usuarios registrados en un mes y año concretos, útil para estadísticas o paneles administrativos.

```
@Query("SELECT u FROM Usuario u WHERE FUNCTION('MONTH', u.fechaInicio) = :mes AND FUNCTION('YEAR', u.fechaInicio) = :anio")
List<Usuario> buscarUsuariosDelMes(@Param("mes") int mes, @Param("anio") int anio);
```

React

La estructura del Frontend es la siguiente:



node_modules → Carpeta con todas las dependencias y paquetes instalados mediante npm.

src → Carpeta principal del código fuente.

Component → Carpeta que guarda los componentes React reutilizables:

- **Footer** → Pie de página.
- **InicioRegistro** → Pantalla o módulo para inicio de sesión y registro.
- **Juegos** → Lógica y vistas relacionadas con los juegos.
- **Mensaje** → Componente para mensajes.
- **Menu** → Menú de navegación.
- **AuthContext.jsx** → Contexto de autenticación

- **ProtectedRoute.jsx** → Componente para proteger rutas (solo accesibles si el usuario está autenticado).

Fotos → Donde guardas imágenes específicas del proyecto.

App.css → Estilos principales de la aplicación.

App.jsx → Componente raíz de la aplicación React.

index.css → Estilos generales aplicados en el index.

main.jsx → Punto de entrada donde se renderiza la app al DOM.

Explicación del archivo `vite.config.js`

El archivo `vite.config.js` es un componente clave del proyecto que se encarga de configurar cómo funciona el entorno de desarrollo del frontend.

En términos generales, este archivo sirve como puente entre el frontend y el backend durante la fase de desarrollo. Por un lado, permite que el proyecto React funcione correctamente, usando herramientas como el plugin de React y el servidor local de Vite, que ofrece recarga rápida cuando haces cambios en el código. Por otro lado, incluye una configuración de proxy que permite redirigir automáticamente las solicitudes que hace el frontend hacia el backend, que normalmente está corriendo en otro puerto. Esto es importante porque evita problemas de comunicación entre ambos lados, especialmente los relacionados con CORS.

```
import { defineConfig } from 'vite';
import react from '@vitejs/plugin-react';

export default defineConfig({
  server: {
    hmr: {
      protocol: 'ws',
      host: 'localhost',
      port: 5173,
      clientPort: 5173,
    },
  },
  proxy: {
    '/api': {
      target: 'http://localhost:8091',
      changeOrigin: true,
      secure: false
    }
  },
  plugins: [react()],
});
```

Librerías principales utilizadas React

React

```
import React, { useContext, useState, useEffect, createContext } from 'react';
```

React es la biblioteca principal sobre la que está construido el proyecto frontend. Permite crear interfaces de usuario dinámicas mediante componentes reutilizables.

Los hooks como `useState`, `useEffect` y `useContext` ayudan a manejar el estado, los efectos secundarios (como llamadas a APIs), y el contexto global de la aplicación (como la autenticación).

react-router-dom

```
import { BrowserRouter as Router, Routes, Route, Navigate, Link, NavLink, useNavigate, useLocation } from 'react-router-dom';
```

Es la librería que permite manejar las rutas en una aplicación React de una sola página (SPA).

Gracias a ella se puede:

- Definir diferentes rutas (`Route`) que cargan componentes según la URL.
- Usar navegación programática (`useNavigate`) para cambiar de página desde el código.
- Redirigir usuarios (`Navigate`) cuando, por ejemplo, no tienen permisos.
- Crear links (`Link`, `NavLink`) que cambian de ruta sin recargar la página.
- Obtener detalles sobre la ruta actual (`useLocation`).

En resumen, `react-router-dom` es esencial para que tu aplicación tenga navegación fluida y se comporte como una app moderna.

axios

```
import axios from 'axios';
```

Axios es una librería para hacer solicitudes HTTP (como GET, POST, PUT, DELETE) hacia un backend o API externa.

La usas principalmente para:

- Pedir datos al backend (como listas, detalles de juegos, usuarios).
- Enviar información (como formularios de registro o login).
- Manejar respuestas y errores de forma sencilla.

Context API (AuthContext)

```
import { AuthContext } from './Component/AuthContext';
```

El Context API de React permite crear un contexto global que se comparte entre componentes sin necesidad de pasar props manualmente.

AuthContext maneja:

- Si el usuario está logueado.
- Los datos del usuario.
- Las funciones para iniciar o cerrar sesión.

Cosas que faltan

d. Pruebas funcionales del sistema, este apartado es fundamental. Debe incluir:

i. Preparación del entorno de pruebas.

ii. Resultados de las pruebas.

e. Errores significativos que hayan ido surgiendo durante el proyecto y cómo se han resuelto.

6. Conclusiones.

a. Este apartado es como un resumen de todo el proyecto.

7. Bibliografía empleada siguiendo el estándar IEEE.

